

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ОТЧЕТ
ПО ЛАБОРАТОРНЫМ РАБОТАМ № 3.1, 3.2
«Проектирование и реализация баз данных в СУБД PostgreSQL»

Обучающийся: Лютый Никита Артемович

Факультет: Прикладной информатики

Группа: К3240

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург
2026

Содержание

1	Описание предметной области (Вариант 14)	2
2	Лабораторная работа № 3.1: Установка и создание БД	2
2.1	Ход выполнения работы	2
2.2	Ответы на контрольные вопросы	3
3	Лабораторная работа № 3.2: Проектирование и заполнение	4
3.1	Физическая реализация модели	4
3.2	Обоснование нормализации	4
3.3	ERD-диаграмма	5
3.4	Листинг кода создания структуры (DDL)	6
3.5	Листинг заполнения данными (DML)	9
3.6	Проверка данных	10
3.7	Резервное копирование	10
4	Выводы	10

1 Описание предметной области (Вариант 14)

Объектом исследования является БД «Служба заказа такси». Система предназначена для автоматизации учета вызовов, распределения заказов между водителями, расчета заработной платы сотрудников и хранения данных о транспортных средствах. Основные требования системы:

- Фиксация вызовов (адрес, время, расстояние, штрафы за ожидание).
- Учет тарифов в зависимости от класса авто и ситуации на дорогах.
- Ведение графиков работы персонала.
- Поддержка различных типов оплаты (наличные, онлайн).

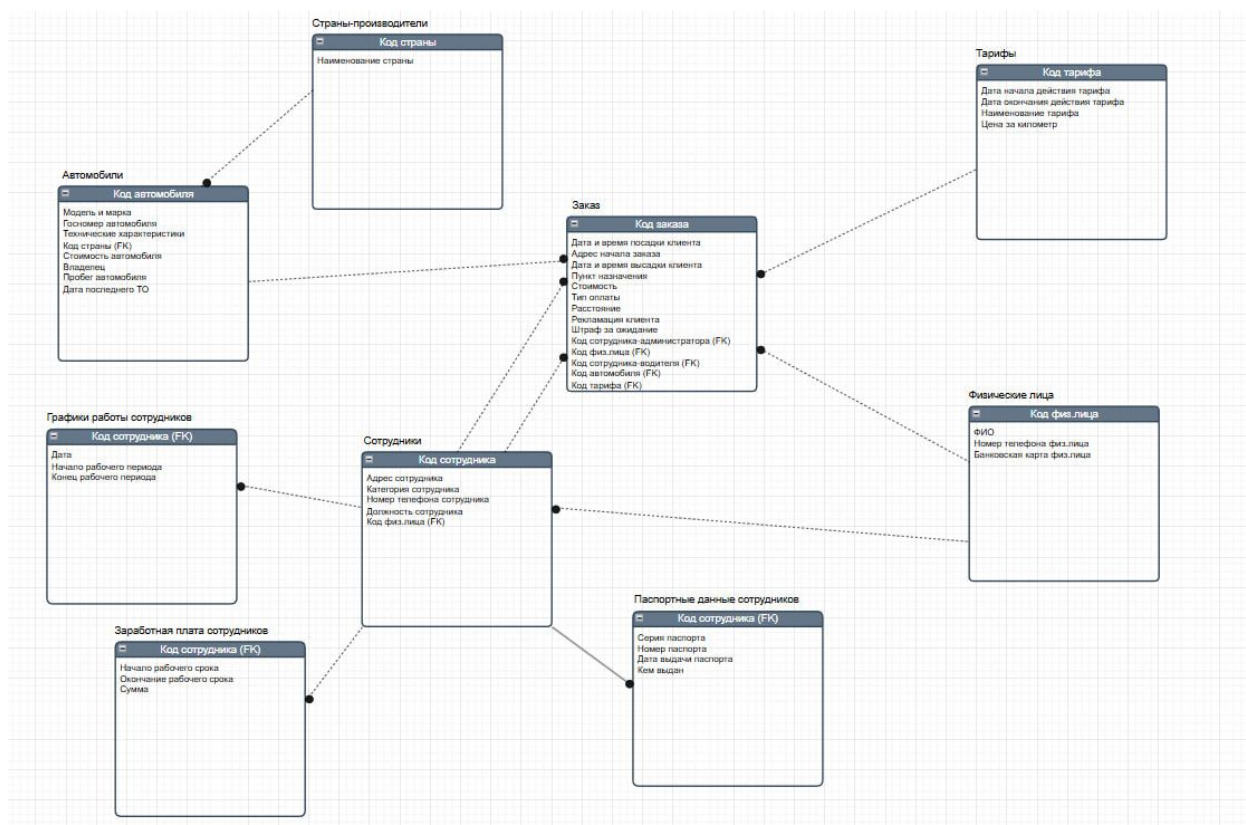


Рис. 1: Схема инфологической модели БД ЛР 2

2 Лабораторная работа № 3.1: Установка и создание БД

2.1 Ход выполнения работы

Для реализации проекта была установлена СУБД PostgreSQL версии 17. С помощью графической оболочки pgAdmin 4 была создана база данных TaxiService. При создании были установлены следующие параметры:

- Владелец: postgres;
- Кодировка: UTF8;
- Региональные параметры: Russian_Russia.1251.

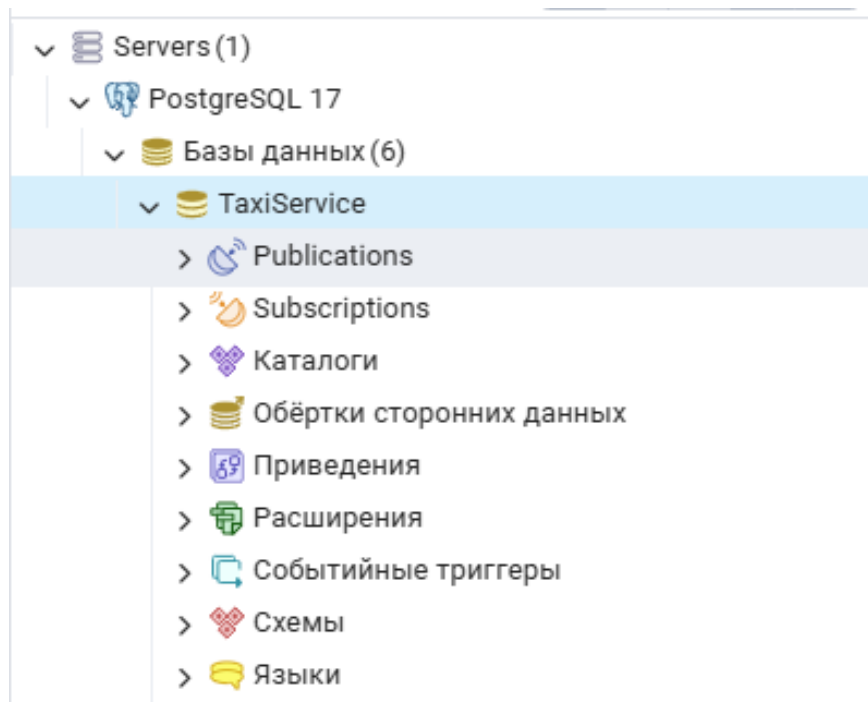


Рис. 2: Обзор объектов с созданной базой данных

2.2 Ответы на контрольные вопросы

1. Какую модель взаимодействия поддерживает СУБД PostgreSQL?

СУБД PostgreSQL поддерживает модель «клиент-сервер». Взаимодействие происходит между серверным процессом, управляющим данными, и клиентским приложением, инициирующим запросы.

2. Из каких взаимодействующих процессов (программ) состоит PostgreSQL-сессия?

Сессия состоит из серверного процесса (программа `postgres`), который управляет файлами базы данных и принимает подключения, и клиентского приложения пользователя (например, `pgAdmin`, `psql` или веб-сервер), которое выполняет операции с базой данных.

3. Какие функции выполняет pgAdmin?

`pgAdmin` представляет собой графическую оболочку с открытым исходным кодом, предназначенную для упрощения создания, обслуживания и использования объектов базы данных. Она предоставляет визуальный интерфейс для администрирования, мониторинга и разработки БД.

4. Какие средства используются в pgAdmin 4 для управления объектами базы данных?

Для управления объектами используются: Обзор объектов (Browser), Инструмент запросов (Query Tool), диалоговые окна свойств (Properties), панели мониторинга (Dashboard), средства обслуживания (Maintenance), а также инструменты резервного копирования (Backup) и восстановления (Restore).

3 Лабораторная работа № 3.2: Проектирование и заполнение

3.1 Физическая реализация модели

На основе инфологической модели (ЛР №2) была разработана физическая структура. **Важное уточнение:** согласно методическим указаниям, составные ключи и ключи типа `UUID` были заменены на суррогатные ключи типа `SERIAL` для упрощения манипуляции данными и обеспечения ссылочной целостности.

Для группировки таблиц была создана схема `taxi`.

3.2 Обоснование нормализации

Реализованная база данных соответствует **третьей нормальной форме (3NF)**:

1. **1NF:** Все значения атрибутов атомарны.
2. **2NF:** Таблицы находятся в 1NF и все неключевые атрибуты функционально зависят от первичного ключа целиком.
3. **3NF:** Таблицы находятся в 2NF и отсутствуют транзитивные зависимости (неключевые поля зависят только от ключа, справочные данные вынесены в отдельные таблицы: `countries`, `tariffs`).

3.3 ERD-диаграмма

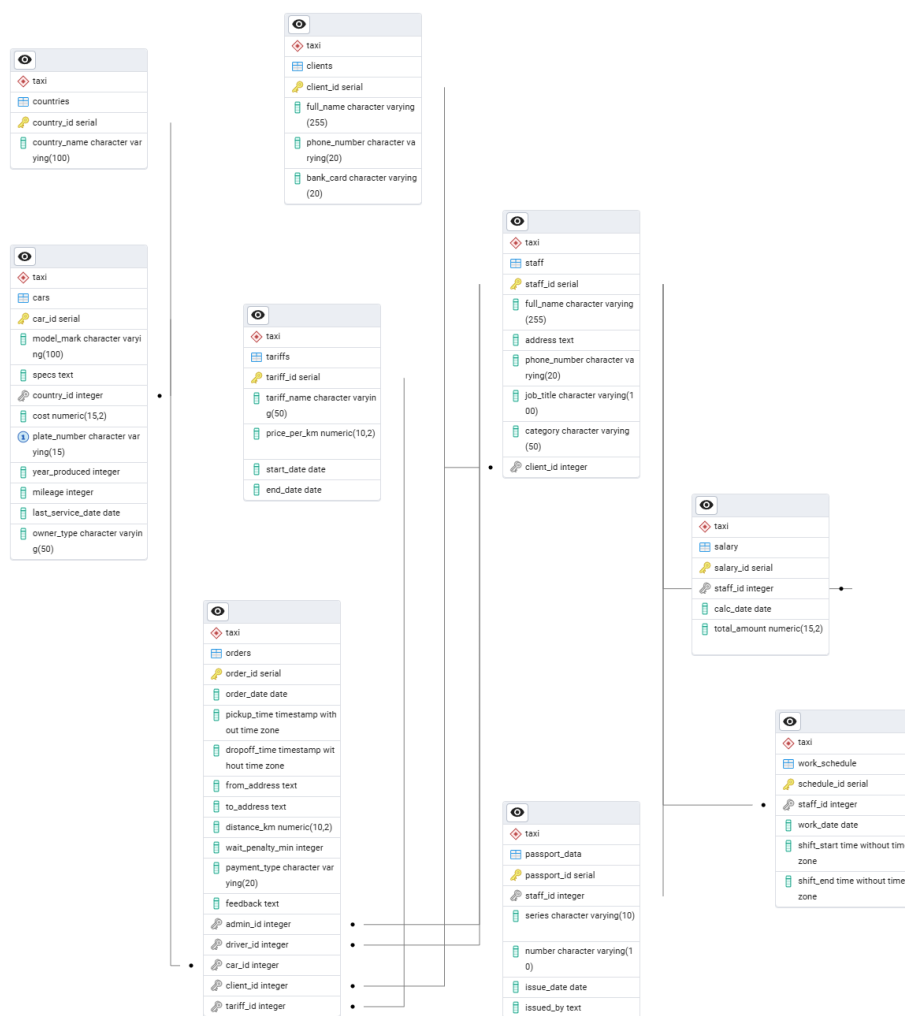


Рис. 3: ERD-диаграмма БД «Служба заказа такси»

3.4 Листинг кода создания структуры (DDL)

Ниже представлен полный SQL-код создания объектов БД и наложения ограничений целостности из отчета ЛР №2.

```
-- 1. Создание схемы
CREATE SCHEMA IF NOT EXISTS taxi;

-- 2. Таблица Стран-производителей
CREATE TABLE taxi.countries (
    country_id SERIAL PRIMARY KEY,
    country_name VARCHAR(100) NOT NULL
);

-- 3. Таблица Тарифов
CREATE TABLE taxi.tariffs (
    tariff_id SERIAL PRIMARY KEY,
    tariff_name VARCHAR(50) NOT NULL,
    price_per_km DECIMAL(10, 2) NOT NULL,
    start_date DATE,
    end_date DATE
);

-- 4. Физические лица
CREATE TABLE taxi.clients (
    client_id SERIAL PRIMARY KEY,
    full_name VARCHAR(255) NOT NULL,
    phone_number VARCHAR(20) NOT NULL,
    bank_card VARCHAR(20)
);

-- 5. Сотрудники
CREATE TABLE taxi.staff (
    staff_id SERIAL PRIMARY KEY,
    full_name VARCHAR(255) NOT NULL,
    address TEXT,
    phone_number VARCHAR(20),
    job_title VARCHAR(100),
    category VARCHAR(50),
    client_id INT REFERENCES taxi.clients(client_id)
);

-- 6. Паспортные данные
CREATE TABLE taxi.passport_data (
    passport_id SERIAL PRIMARY KEY,
    staff_id INT UNIQUE REFERENCES taxi.staff(staff_id),
    series VARCHAR(10),
    number VARCHAR(10),
    issue_date DATE,
    issued_by TEXT
);

-- 7. Автомобили
CREATE TABLE taxi.cars (
```

```

    car_id SERIAL PRIMARY KEY,
    model_mark VARCHAR(100) NOT NULL,
    specs TEXT,
    country_id INT REFERENCES taxi.countries(country_id),
    cost DECIMAL(15, 2),
    plate_number VARCHAR(15) UNIQUE,
    year_produced INT,
    mileage INT,
    last_service_date DATE,
    owner_type VARCHAR(50)
);

-- 8. График работы
CREATE TABLE taxi.work_schedule (
    schedule_id SERIAL PRIMARY KEY,
    staff_id INT REFERENCES taxi.staff(staff_id),
    work_date DATE NOT NULL,
    shift_start TIME,
    shift_end TIME
);

-- 9. Заказы
CREATE TABLE taxi.orders (
    order_id SERIAL PRIMARY KEY,
    order_date DATE DEFAULT CURRENT_DATE,
    pickup_time TIMESTAMP,
    dropoff_time TIMESTAMP,
    from_address TEXT,
    to_address TEXT,
    distance_km DECIMAL(10, 2),
    wait_penalty_min INT DEFAULT 0,
    payment_type VARCHAR(20),
    feedback TEXT,
    admin_id INT REFERENCES taxi.staff(staff_id),
    driver_id INT REFERENCES taxi.staff(staff_id),
    car_id INT REFERENCES taxi.cars(car_id),
    client_id INT REFERENCES taxi.clients(client_id),
    tariff_id INT REFERENCES taxi.tariffs(tariff_id),
    CONSTRAINT chk_wait_penalty CHECK (wait_penalty_min >= 0)
);

-- 10. Заработная плата
CREATE TABLE taxi.salary (
    salary_id SERIAL PRIMARY KEY,
    staff_id INT REFERENCES taxi.staff(staff_id),
    calc_date DATE,
    total_amount DECIMAL(15, 2)
);

-- 11. Ограничения для таблицы Автомобили
ALTER TABLE taxi.cars
    ADD CONSTRAINT chk_plate_format CHECK (plate_number ~ '
    АЯАЯ^[-][0-9]{3}[-]{2}[0-9]{2,3}$'),

```



```

    ADD CONSTRAINT chk_car_cost CHECK (cost > 0),
    ADD CONSTRAINT chk_car_mileage CHECK (mileage >= 0),
    ADD CONSTRAINT chk_service_date CHECK (last_service_date >= '
2000-01-01');

-- 12. Ограничения для таблицы Сотрудники
ALTER TABLE taxi.staff
    ADD CONSTRAINT chk_staff_phone CHECK (phone_number ~ '^\\+7[0-9]{10}
$'),
    ADD CONSTRAINT chk_job_title CHECK (job_title IN ('Администратор',
'Водитель'));

-- 13. Ограничения для таблицы Физические лица
ALTER TABLE taxi.clients
    ADD CONSTRAINT chk_client_phone CHECK (phone_number ~ '
^\\+7[0-9]{10}$'),
    ADD CONSTRAINT chk_bank_card CHECK (bank_card ~ '^[0-9]{16,19}$');

-- 14. Ограничения для таблицы Паспортные данные
ALTER TABLE taxi.passport_data
    ADD CONSTRAINT chk_pass_series CHECK (series ~ '^[0-9]{4}$'),
    ADD CONSTRAINT chk_pass_number CHECK (number ~ '^[0-9]{6}$'),
    ADD CONSTRAINT chk_pass_date CHECK (issue_date <= CURRENT_DATE);

-- 15. Ограничения для таблицы Тарифы
ALTER TABLE taxi.tariffs
    ADD CONSTRAINT chk_tariff_price CHECK (price_per_km > 0),
    ADD CONSTRAINT chk_tariff_dates CHECK (end_date IS NULL OR end_date
>= start_date);

-- 16. Ограничения для таблицы График работы
ALTER TABLE taxi.work_schedule
    ADD CONSTRAINT chk_shift_time CHECK (shift_end > shift_start);

-- 17. Ограничения для таблицы Заработная плата
ALTER TABLE taxi.salary
    ADD CONSTRAINT chk_salary_amount CHECK (total_amount >= 0);

-- 18. Ограничения для таблицы Заказы
ALTER TABLE taxi.orders
    ADD CONSTRAINT chk_order_distance CHECK (distance_km > 0),
    ADD CONSTRAINT chk_order_payment CHECK (payment_type IN ('Наличные'
, 'Онлайн')),
    ADD CONSTRAINT chk_order_times CHECK (dropoff_time IS NULL OR
dropoff_time >= pickup_time);

```

Листинг 1: Полный скрипт реализации структуры и данных

3.5 Листинг заполнения данными (DML)

Для демонстрации работоспособности БД был произведен рефакторинг тестовых данных и вставка новых записей.

```
-- Добавляем страны
INSERT INTO taxi.countries (country_name) VALUES ('Россия'), ('
    Германия'), ('Япония');

-- Добавляем тарифы
INSERT INTO taxi.tariffs (tariff_name, price_per_km) VALUES ('Эконом',
    25.0), ('Комфорт', 45.0), ('Бизнес', 80.0);

-- Добавляем пассажиров
INSERT INTO taxi.clients (full_name, phone_number, bank_card) VALUES
('Иванов ИИ..', '89001112233', '4444555566667777'),
('Петров ПП..', '89005556677', NULL);

-- Добавляем сотрудников
INSERT INTO taxi.staff (full_name, job_title, category) VALUES
('Смирнова АВ..', 'Администратор', 'Высшая'),
('Быков ДС..', 'Водитель', '1 класс');

-- Добавляем авто
INSERT INTO taxi.cars (model_mark, plate_number, country_id, owner_type
    ) VALUES
('Hyundai Solaris', 'APT12377', 1, 'Компания'),
('Mercedes E-Class', '00077799', 2, 'Таксист');

-- Добавляем тестовый заказ
INSERT INTO taxi.orders (from_address, to_address, distance_km,
    payment_type, driver_id, car_id, client_id, tariff_id)
VALUES ('ул. Ленина 1', 'аэропорт Домодедово', 45.5, 'Онлайн', 2, 2, 1,
    3);
```

Листинг 2: Заполнение базы данных

3.6 Проверка данных

После выполнения запросов был осуществлен визуальный контроль содержимого таблиц.

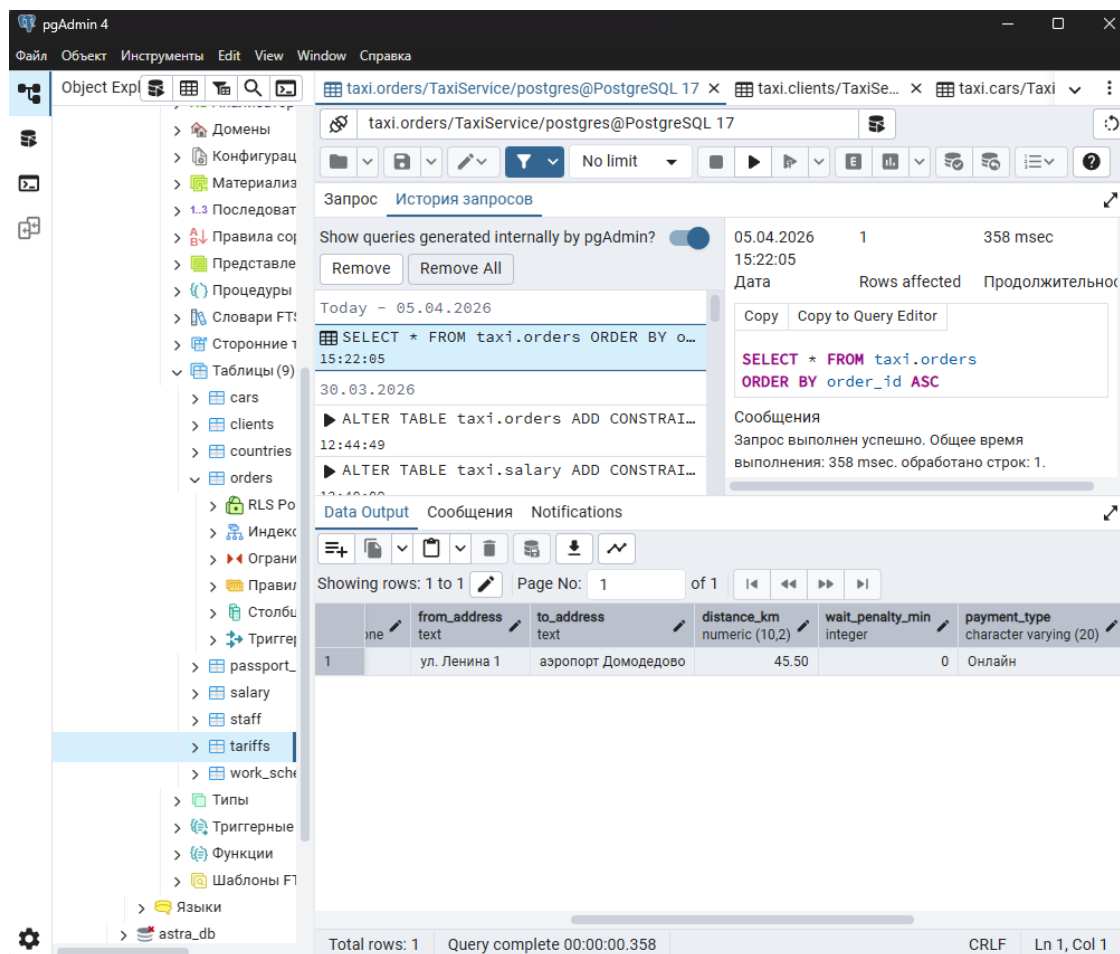


Рис. 4: Визуализация наполненной таблицы заказов

3.7 Резервное копирование

В соответствии с требованиями ЛР №3.2 были созданы два дампа:

1. **taxi_custom.backup** — для восстановления через pgAdmin.
2. **taxi_plain.sql** — для анализа SQL-кода.

В дампы включена команда `CREATE DATABASE` для обеспечения автономности восстановления.

4 Выводы

В ходе выполнения лабораторных работ №3.1 и 3.2 была успешно развернута СУБД PostgreSQL и реализована физическая модель базы данных «Служба заказа такси». Процесс разработки включал в себя создание схем, таблиц, настройку связей Foreign Key и внедрение комплексных ограничений CHECK для валидации данных на уровне СУБД. Проведена проверка целостности данных и освоен механизм резервного копирования.